SIMATS ENGINEERING

CO5-ASSESSMENT TOOL3 - Reverse Engineering

COURSE NAME: **Artificial Intelligence** **COURSE CODE: CSA1709**

Register Number	**X. Joshua**
Name	**192312188**

COURSE OUTCOME COVERED

CO5: *Design intelligent agents and real-time AI-based systems (chatbots, expert systems, emotion detection, edge AI, autonomous drones, and related applications) by selecting appropriate agent architectures, knowledge representation, and reasoning mechanisms to solve real-world problems. (BL6)*

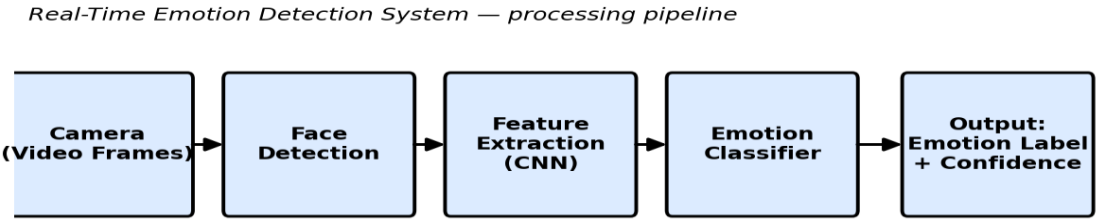
Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Q1. The following is a conversation between a user and an AI-based flight-booking chatbot. Study the transcript given below and reverse-engineer the chatbot's design by: (a) writing its PEAS description, (b) classifying the type of agent it represents with justification, (c) identifying the likely component (NLU, Dialogue Manager, Knowledge Base, or NLG) responsible for each chatbot response, and (d) suggesting one enhancement using a learning-agent architecture.

Speaker	Message
User	I want to book a flight to Chennai next Friday.
Chatbot	Sure! What time would you like to depart, and do you have a preferred airline?
User	Morning flight, no preference.
Chatbot	I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?

Q2. The block diagram below shows the processing pipeline of a Real-Time Emotion Detection System. Reverse-engineer the system by: (a) explaining the function performed at each stage of the pipeline, (b) identifying the likely AI/ML technique used at each stage, (c) writing the PEAS description of the system, and (d) classifying the agent type (e.g., simple reflex, model-based) with justification.



Q3. An Expert System Shell for medical diagnosis contains the rule base, input facts, and reasoning trace given below. Study it and reverse-engineer the shell by: (a) identifying the type of chaining (forward/backward) used to

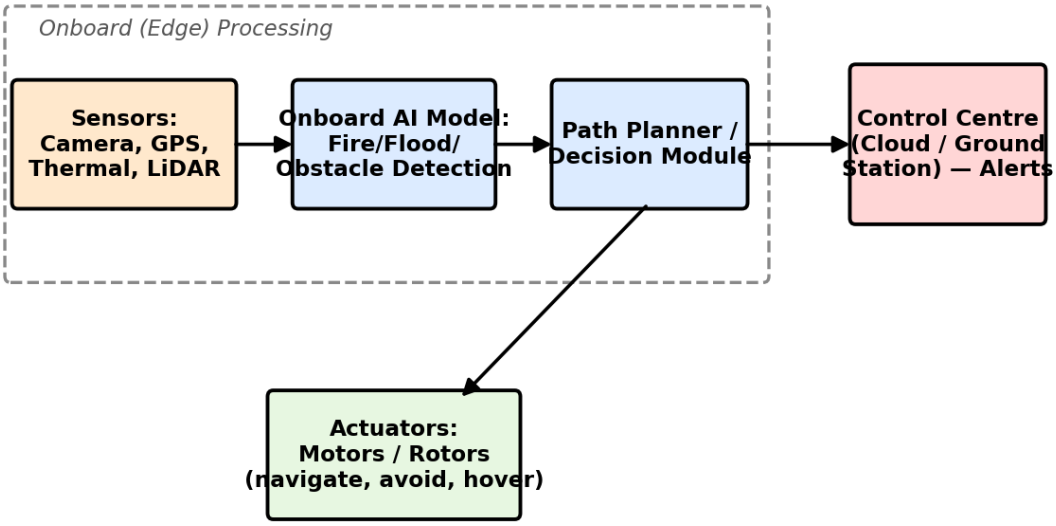
reach the diagnosis, with justification, (b) identifying the expert-system-shell components illustrated (knowledge base, inference engine, explanation facility), (c) explaining how the shell would perform knowledge acquisition to add a new rule for dengue, and (d) explaining, with reasoning, whether backward chaining could reach the same diagnosis.

Rule	IF (condition)	THEN (conclusion)
R1	fever = yes AND cough = yes	suspect = flu
R2	suspect = flu AND body_ache = yes	diagnosis = Influenza
R3	fever = yes AND rash = yes	suspect = measles

Given facts: fever = yes, cough = yes, body_ache = yes
Reasoning trace: R1 fires first (fever & cough matched) → suspect = flu. R2 then fires (suspect = flu & body_ache matched) → diagnosis = Influenza.
Explanation generated by the shell: *"Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza."*

Q4. The diagram below shows the system architecture of an Autonomous Drone used for Disaster Detection with Edge AI deployment. Reverse-engineer the design by: (a) writing the PEAS description of the drone agent, (b) classifying the agent type (goal-based/utility-based) with justification, (c) identifying which components run on the edge versus the cloud and justifying why on-edge processing was chosen for detection and obstacle avoidance, and (d) identifying whether the agent's control architecture is reactive, deliberative, or hybrid, with justification.

Autonomous Drone — Disaster Detection (Edge AI) system architecture



Q5. The table below shows sample output logs from an AI-based Deepfake & Face Authenticity Verification system across five video frames. Study the output and reverse-engineer the system by: (a) identifying the likely input features used by the model (e.g., blink rate, texture/artifact score), (b) identifying the type of model/technique likely used to produce these outputs, (c) explaining how the final Real/Fake decision may have been derived from the frame-level confidence scores, and (d) suggesting one way a swarm/ensemble-based approach could improve the system's reliability.

Frame No.	Blink Rate (per 10s)	Texture/Artifact Score	Fake Confidence (%)	Prediction
1	4	0.12	8	Real
2	1	0.61	74	Fake
3	0	0.83	91	Fake
4	5	0.09	5	Real
5	2	0.55	68	Fake

Overall system verdict: Fake (3 of 5 frames flagged — majority vote)
Code of Conduct:

I X. Joshua (192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
X. Joshua

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Identification of Agent Type & PEAS (correctly inferred from the given artifact)	Correctly identifies the agent type and gives a complete, accurate PEAS description, fully consistent with the given artifact.	Identifies the agent type and PEAS description correctly with only minor omissions.	Partially correct identification of agent type/PEAS; some elements are missing or inconsistent with the artifact.	Agent type/PEAS identification is largely incorrect or not attempted.	**10**
Identification of Architecture & Components (mapping artifact evidence to system components)	Accurately identifies all relevant architectural components and correctly maps each to specific evidence in the artifact.	Identifies most components correctly with reasonable mapping to the artifact; minor gaps present.	Identifies some components; mapping to the artifact is weak or partially incorrect.	Little or no correct identification of architecture/components.	**10**
Identification of Underlying Technique/Algorithm/Reasoning Mechanism	Correctly reverse-engineers the technique, algorithm, or reasoning mechanism (e.g., chaining type, ML technique) with clear supporting evidence.	Correctly identifies the technique/mechanism with adequate, mostly accurate justification.	Identification is only partially correct or lacks sufficient supporting evidence.	Technique/mechanism is incorrectly identified or not addressed.	**10**
Quality of Justification & Use of Evidence (linking conclusions back to the given artifact)	Every conclusion is clearly justified with specific, accurate references to the given artifact.	Most conclusions are justified with reasonable references to the artifact.	Justification is vague or only loosely connected to the artifact.	Conclusions are stated with little or no justification or evidence.	**10**
Clarity & Completeness of Presentation (addresses all sub-parts of the question in an	All sub-parts are addressed completely, in a clear, well-organised, and logically	Most sub-parts are addressed clearly with minor gaps in organisation	Some sub-parts are missing or the presentation is unclear/poorly organised.	Response is incomplete, disorganised, or largely missing.	**10**

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
organised manner)	structured manner.	or completeness.			
				Total	50

Question 1: Flight-Booking Chatbot

Transcript reference - Turn 1: User asks to book a flight to Chennai next Friday; Chatbot asks for departure time and airline preference. Turn 2: User replies morning flight, no preference; Chatbot reports 3 morning flights found and offers the cheapest first.

(a) PEAS Description

PEAS Element	Description
Performance measure	Successful flight booking, correct slot filling (destination, date, time, airline), minimum clarification turns, user satisfaction
Environment	The user (via chat), the airline reservation/flight-availability database, a multi-turn text conversation (partially observable, sequential, stochastic since user replies are unpredictable)
Actuators	Generated chat replies (clarifying questions, flight results); booking/search calls made to the reservation system
Sensors	The text input parser that receives and reads each user message

(b) Agent Type Classification

This chatbot is a goal-based agent with model-based (internal state) tracking. In Turn 2 it responds only about the still-missing slots (time and airline) instead of re-asking for destination or date, which shows it retained the destination = Chennai and date = next Friday extracted in Turn 1. A simple reflex agent has no memory of earlier input and could not do this. The chatbot is also clearly working toward a goal - a completed booking - since every response either requests a missing piece of information or moves the user closer to selecting a flight, rather than reacting to each message in isolation.

(c) Component Responsible for Each Chatbot Response

Chatbot Response	Component	Evidence
"Sure! What time would you like to depart, and do you have a preferred airline?"	NLU + Dialogue Manager + NLG	NLU extracted destination = Chennai and date = next Friday from the user's first message; the Dialogue Manager checked its slot-filling state, found time and airline still missing, and chose a request-info action; NLG phrased that action as the question shown
"I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?"	NLU + Knowledge Base + Dialogue Manager + NLG	NLU extracted time = morning from the user's second message; the Knowledge Base (flight database) was queried with destination/date/time and returned 3 matching flights; the Dialogue Manager decided to report the

(d) Enhancement Using a Learning-Agent Architecture

A Learning Element could be added alongside the existing performance element, using this user's past bookings (e.g. a history of choosing morning flights and one particular airline) as training data. A Critic would compare predicted preferences against the user's actual choices to generate feedback, and a Problem Generator would occasionally offer a non-default option (a different airline) to keep exploring the user's true preferences. Over time this would let the chatbot pre-fill or skip the "preferred airline" question for returning users, reducing the number of clarification turns shown in this transcript.

Question 2: Real-Time Emotion Detection System

Pipeline shown in the diagram: Camera (Video Frames) -> Face Detection -> Feature Extraction (CNN) -> Emotion Classifier -> Output: Emotion Label + Confidence.

(a) & (b) Function and Likely AI/ML Technique at Each Stage

Stage	Function Performed	Likely AI/ML Technique
Camera (Video Frames)	Captures continuous raw video input, supplying frame-by-frame images to the pipeline	Sensor/image acquisition - not an AI/ML step
Face Detection	Locates and crops the face region within each frame, discarding background	Classical/light-weight detector, e.g. Haar-cascade, HOG+SVM, or a small CNN detector such as MTCNN
Feature Extraction (CNN)	Converts the cropped face into a numeric feature representation capturing shape and texture cues (eyebrows, mouth, eyes)	Convolutional Neural Network - convolution and pooling layers, as explicitly labelled in the diagram
Emotion Classifier	Maps the extracted feature vector to one of a fixed set of emotion categories	Fully-connected softmax layer (or a separate classifier such as SVM/Random Forest on top of the CNN features)
Output: Emotion Label + Confidence	Reports the predicted emotion along with a probability/confidence score	Softmax probability read out as the confidence value

(c) PEAS Description

PEAS Element	Description
Performance measure	Classification accuracy, correct face detection, low latency for real-time use, robustness to lighting/pose changes
Environment	Live video stream of human faces under variable lighting, pose and movement (partially observable, continuous, dynamic)
Actuators	Display of the emotion label and confidence score to a screen, dashboard, or downstream application
Sensors	Camera capturing the video frames

(d) Agent Type Classification

This is a simple reflex agent. The diagram shows a single, fixed forward pipeline - camera to face detection to feature extraction to classification to output - with no branch or box representing stored history from earlier frames feeding back into a later decision. Each frame is mapped directly to an emotion label through the same condition-action sequence, which is exactly the behaviour of a simple reflex agent rather than a model-based agent (which would need an explicit internal-state block, not shown here, to track emotional trends across frames).

Question 3: Expert System Shell for Medical Diagnosis

Given rules: R1 (fever=yes AND cough=yes -> suspect=flu), R2 (suspect=flu AND body_ache=yes -> diagnosis=Influenza), R3 (fever=yes AND rash=yes -> suspect=measles). Given facts: fever=yes, cough=yes, body_ache=yes. Trace: R1 fires, then R2 fires, giving diagnosis = Influenza.

(a) Type of Chaining Used

Forward chaining was used. The reasoning trace starts from the known facts (fever = yes, cough = yes, body_ache = yes) and fires whichever rule's IF-conditions are already satisfied - R1 first, because fever and cough matched - producing the new fact suspect = flu, which then satisfies R2 and produces diagnosis = Influenza. This is data-driven reasoning that moves from facts toward a conclusion, which is the definition of forward chaining, as opposed to starting from a hypothesis and working backward.

(b) Expert-System-Shell Components Illustrated

Component	Evidence in the Given Artifact
Knowledge Base	The rule set R1-R3 together with the given facts (fever=yes, cough=yes, body_ache=yes)
Inference Engine	The rule-matching and firing mechanism shown in the reasoning trace: it evaluated which rules' IF-conditions were met and fired R1, then R2, in that order
Explanation Facility	The generated sentence "Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza", which translates the firing sequence into a human-readable justification

(c) Knowledge Acquisition for a New Dengue Rule

A domain expert (physician) would supply the diagnostic criteria for dengue, for example fever = yes AND rash = yes AND joint_pain = yes -> suspect = dengue. A knowledge engineer would encode this in the same IF-THEN production-rule syntax already used by R1-R3, adding joint_pain as a new fact attribute that the data-entry form/UI must also be able to capture. Because R3 for measles also fires on fever + rash, the rule base would need to be checked for this overlap - for instance by requiring joint_pain to distinguish dengue from measles - before the new rule is inserted. Once validated, the rule is simply added to the knowledge base; the inference engine itself needs no changes, since it works generically over whatever rules exist.

(d) Could Backward Chaining Reach the Same Diagnosis?

Yes. Backward chaining would start from the goal diagnosis = Influenza and look at R2 to see what is required to prove it: suspect = flu and body_ache = yes. It would then treat suspect = flu as a sub-goal and look at R1 to see what proves that: fever = yes and cough = yes. Finally it checks the given facts and confirms fever = yes, cough = yes and body_ache = yes are all true, so the goal is proved and diagnosis = Influenza is reached - the same result as forward chaining. The difference is efficiency and direction: forward chaining fired every rule whose conditions happened to match the facts, while backward chaining would only check the specific facts needed to prove the Influenza hypothesis, which is more efficient when testing one particular diagnosis rather than deriving all possible conclusions.

Question 4: Autonomous Drone for Disaster Detection (Edge AI)

Diagram shows an Onboard (Edge) Processing boundary containing Sensors (Camera, GPS, Thermal, LiDAR) -> Onboard AI Model (Fire/Flood/Obstacle Detection) -> Path Planner/Decision Module, which connects to Actuators (Motors/Rotors) and, outside the edge boundary, to a Control Centre (Cloud/Ground Station) that receives Alerts.

(a) PEAS Description

PEAS Element	Description
Performance measure	Accurate and fast fire/flood detection, successful obstacle avoidance, safe navigation, timely alerts to the control centre, efficient battery use
Environment	Disaster-affected outdoor terrain with obstacles and changing weather (partially observable, dynamic, continuous, stochastic)
Actuators	Motors/rotors that navigate, avoid obstacles and hover; the wireless link that sends alerts to the control centre
Sensors	Camera, GPS, thermal sensor, LiDAR

(b) Agent Type Classification

This is a utility-based agent. The presence of a dedicated Path Planner / Decision Module, rather than a single goal-check, implies the drone is choosing among several possible paths that could all technically reach the disaster area - it must rank them by a utility function that trades off distance, safety margin from LiDAR-detected obstacles, and battery/hover efficiency. A purely goal-based agent would only ask whether a path reaches the goal, not which path is best, so the explicit planning module points to utility-based decision-making.

(c) Edge vs Cloud Components

Runs on the Edge (onboard)	Runs on the Cloud/Ground
Sensors: Camera, GPS, Thermal, LiDAR	Control Centre (Cloud/Ground Station) - receives Alerts only
Onboard AI Model: Fire/Flood/Obstacle Detection	
Path Planner / Decision Module	

On-edge processing was chosen for detection and obstacle avoidance because these are safety-critical, real-time decisions that cannot tolerate network round-trip delay or the possibility of lost connectivity, which is common in disaster zones. Keeping detection and avoidance onboard also means the drone stays operational if the link to the control centre drops, and it reduces bandwidth use since only summarised alerts, not raw sensor streams, need to be transmitted to the cloud.

(d) Control Architecture Classification

The drone uses a hybrid (reactive + deliberative) control architecture. The Onboard AI Model performing fire/flood/obstacle detection and feeding directly into avoidance behaviour represents the reactive layer,

needed for fast, real-time obstacle response. The Path Planner/Decision Module represents the deliberative layer, responsible for higher-level route planning toward the disaster zone and deciding when to alert the control centre. Both layers appear as separate, connected modules in the diagram, which is the defining feature of a hybrid architecture rather than a purely reactive or purely deliberative one.

Question 5: Deepfake & Face Authenticity Verification System

Given output across 5 frames: Frame 1 (blink 4, texture 0.12, fake conf 8%, Real), Frame 2 (blink 1, texture 0.61, fake conf 74%, Fake), Frame 3 (blink 0, texture 0.83, fake conf 91%, Fake), Frame 4 (blink 5, texture 0.09, fake conf 5%, Real), Frame 5 (blink 2, texture 0.55, fake conf 68%, Fake). Overall verdict: Fake (3 of 5 frames flagged - majority vote).

(a) Likely Input Features Used by the Model

The two features explicitly logged are blink rate per 10 seconds, a behavioural/physiological cue (deepfake generators are typically trained on more open-eye frames and under-represent natural blinking, so an unusually low blink rate is suspicious), and a texture/artifact score, a visual cue capturing unnatural skin texture or blending artifacts typical of GAN-generated or face-swapped frames. The data shows both features moving together - low blink rate and high texture score in Frames 2, 3 and 5 correspond to high fake confidence, while high blink rate and low texture score in Frames 1 and 4 correspond to low fake confidence - indicating the model combines both features to compute its confidence score.

(b) Likely Model/Technique Used

Producing a continuous, probability-like Fake Confidence percentage from multiple input features is characteristic of a supervised binary classification model, most likely a CNN-based deepfake detector (in the style of XceptionNet) for the texture/artifact score, combined with a temporal model (such as an eye-aspect-ratio measure feeding an LSTM) for the blink-rate cue, with the two outputs fused - for example by a logistic regression or a weighted-sum layer - into the single per-frame Fake Confidence value shown in the table.

(c) How the Final Real/Fake Verdict Was Derived

Frame	Fake Confidence	Frame-Level Flag (threshold 50%)
1	8%	Real
2	74%	Fake
3	91%	Fake
4	5%	Real
5	68%	Fake

Each frame's Fake Confidence appears to have first been thresholded independently, at roughly 50%, to produce a per-frame Real/Fake flag - Frames 2, 3 and 5 exceed 50% and are flagged Fake, while Frames 1 and 4 fall well below it and are flagged Real. The system then combined the five per-frame flags with a simple majority-vote rule, and since 3 of the 5 frames were flagged Fake, the overall system verdict was reported as Fake, exactly as stated in the given output.

(d) Improving Reliability with a Swarm/Ensemble Approach

Instead of one model's score being thresholded and then majority-voted across frames, several independent detector agents could be run in parallel - for example one specialised in blink/temporal cues, one in texture/artifact analysis, and one in lip-sync consistency - each producing its own vote or confidence score. An ensemble/swarm aggregation (such as weighted voting, where agents that have proven more reliable on past videos carry more weight, or a particle-swarm-optimised weighting scheme) would combine these independent judgements into the final verdict. This would make the system less dependent on any single feature such as texture score, which could be unreliable under poor lighting or compression, and would therefore be more robust than the current two-feature, unweighted majority vote.